

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Бочарников Максим

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург
2026 г.

1. Цель работы

Реализовать переход от монолитного backend-решения к микросервисной архитектуре, настроить межсервисное взаимодействие, контейнеризировать сервисы, организовать ручное и автоматизированное развёртывание, а также обеспечить воспроизводимость поставки через CI/CD.

2. Выполненные задачи по ТЗ

2.1 ЛР2: Реализация микросервисов

В проекте выполнено разделение на независимые сервисы:

- tagmentor-backend — основной API, бизнес-логика, работа с БД, авторизация, управление проектами/исследованиями/файлами.
- tagmentor-ml — отдельный ML-сервис для задач генерации и классификации.
- tagmentor-scheduler — фоновая обработка задач (job worker), вынесенная в отдельный runtime-процесс.
- tagmentor-frontend — отдельный клиентский сервис.
- инфраструктурные сервисы: PostgreSQL, Redis, стек мониторинга (Prometheus, Loki, Promtail, Grafana).

Таким образом, система реализована как набор слабо связанных сервисов с разделением ответственности и независимыми жизненными циклами.

2.2 ДЗ5: Межсервисное взаимодействие через очереди сообщений

В рамках текущей реализации асинхронное взаимодействие построено по событийной модели:

- используется Redis Pub/Sub для публикации и доставки событий (в т.ч. уведомлений в SSE-каналы);
- backend ведёт фоновые задачи через таблицу job_files и отдельный worker-процесс (tagmentor-scheduler);
- backend отправляет задания в ML-сервис по HTTP;
- ML-сервис возвращает результат через callback-эндпоинты backend (/api/v1/callback/...).

Это обеспечивает:

- неблокирующую обработку долгих операций;
- изоляцию вычислительной нагрузки ML;
- устойчивость основного API под нагрузкой.

2.3 ЛР3: Docker и docker-compose

Для каждого сервиса подготовлены отдельные Dockerfile:

- backend/Dockerfile.cicd
- ml/Dockerfile.cicd
- frontend/Dockerfile.cicd

Собран общий docker-compose.cicd.yml, который описывает:

- сборку и запуск всех сервисов;
- внутреннюю сеть TAGMENTOR_net;
- порядок запуска через depends_on и healthcheck;
- маршрутизацию через Traefik labels;
- постоянные volume для PostgreSQL и monitoring-стека;
- переменные окружения через env/*.env.

Итог: поднятие всей системы выполняется декларативно и воспроизводимо из одного compose-конфига.

2.4 ЛР4: Развёртывание на удалённом сервере (ручное)

Ручной сценарий развёртывания включает:

1. Подготовку сервера (Docker, Docker Compose, доступ по SSH).
2. Копирование docker-compose.cicd.yml и env-файлов на хост.
3. Запуск сервисов через docker compose up -d.
4. Проверку доступности:
 - backend API,
 - frontend,
 - ML-сервис,
 - мониторинг (Grafana/Prometheus/Loki).
5. Проверку сквозного пользовательского сценария (авторизация → создание сущностей → запуск ML-задачи → получение callback-результата).

2.5 ДЗ6: CI/CD (автодеплой)

В проекте настроен GitLab CI/CD (.gitlab-ci.yml) с инфраструктурной интеграцией:

- используется общий pipeline-шаблон из корпоративного DevOps-репозитория;
- заданы переменные для registry и удалённого deploy:
 - REGISTRY_HOST=harbor.metamentor.pro
 - REMOTE_PROJECT_PATH=/opt/compose/tagmentor
 - SOURCE_DOCKER_COMPOSE_FILE=./docker-compose.cicd.yml
- реализованы отдельные build-джобы:
 - build_backend
 - build_frontend
 - build_ml
- сборка выполняется с кэшированием через Kaniko;
- образы публикуются в Harbor (tagmentor/backend, tagmentor/frontend, tagmentor/ml);
- деплой запускается триггерно при обновлении целевой ветки, без ручного вмешательства.

Итог: обеспечен полный непрерывный цикл поставки — от коммита до обновления удалённого окружения.

3. Архитектурные преимущества реализованного решения

- Масштабируемость: backend, ml и scheduler масштабируются независимо.
- Изоляция ответственности: бизнес-API, ML-обработка и фоновая оркестрация разделены.
- Устойчивость: долгие операции вынесены из request-response цикла.
- Наблюдаемость: централизованные метрики и логи.
- Повторяемость поставки: инфраструктура описана декларативно, окружение воспроизводимо.
- Готовность к росту: архитектура уже пригодна для перехода на RabbitMQ/Kafka как выделенный брокер сообщений без ломки API-контрактов.

4. Проверка работоспособности (тест-сценарий для отчёта)

1. Выполнить логин в системе и получить доступ к защищённым endpoint.
2. Создать проект/исследование через backend API.
3. Отправить задачу на классификацию/генерацию в ML-процесс.
4. Проверить асинхронный статус задачи.
5. Проверить доставку результатов через callback в backend.
6. Проверить клиентское уведомление через SSE (Redis Pub/Sub).
7. Проверить логи и метрики сервисов в мониторинге.
8. Выполнить повторный деплой через CI и убедиться в стабильности сценария.

5. Результаты

- Выполнено реальное микросервисное разделение приложения.
- Реализовано асинхронное межсервисное взаимодействие.
- Подготовлены Docker-образы каждого сервиса и единый docker-compose.
- Настроено ручное и автоматическое развёртывание на удалённую среду.
- Подключён промышленный CI/CD-контур с публикацией образов и автодеплоем.

6. Вывод

В ходе выполнения блока лабораторных работ реализована зрелая микросервисная платформа: сервисы декомпозированы по ответственности, межсервисные коммуникации переведены в асинхронный режим, доставка обновлений автоматизирована через CI/CD, а эксплуатационный контур дополнен мониторингом и централизованным логированием.

Решение соответствует практикам production-разработки и готово к дальнейшему масштабированию.